

Scripting

Huiswerk & Zelfstudie | Les 1 & Les 2

Wat is zelfstudie?

Zelfstudie betekent dat je zelfstandig nieuwe stof verkent, buiten de les om. Je gebruikt daarvoor W3Schools, een gratis leerplatform voor programmeren. Ga naar → w3schools.com/python en werk de volgende twee onderdelen door:

- **Python Get Started**
- **Python Syntax**

Mappenstructuur

Maak op je pc een nette mapstructuur aan voor al je Python-opdrachten. Zo houd je alles overzichtelijk:

```
Documents/  
└─ Python/  
    └─ Les_1/  
        │   opdracht1.py  
        │   opdracht2.py  
        │   opdracht3.py  
        └─ Les_2/  
            │   opdracht1.py  
            │   opdracht2.py  
            │   opdracht3.py  
            │   opdracht4.py  
            └─ Uitdagend/  
                opdracht1.py
```

Let op: lever per opdracht een los .py bestand in.

Geef elk bestand een duidelijke naam, bijv. opdracht1.py, opdracht2.py, enz.

Les 1

Het inleverpunt gaat deze week open. Les 1 en les 2 lever je tegelijk in. Dit zijn de opdrachten van les 1.

Let op: voorzie elk stuk code van commentaar (#).

Opdracht 1: Lists & Loops

Maak een nieuw Python-bestand `opdracht1.py` en voer het volgende uit:

- maak zelf een list met 5 zelfgekozen getallen en noteer ze in je bestand
- tel hoeveel items erin zitten
tip: `len()`
- bereken de som van alle getallen
tip: `sum()`
- bereken de gemiddelde waarde
- print alle resultaten netjes met f-strings

Opdracht 2: Dictionaries

Maak een nieuw Python-bestand `opdracht2.py` en voer het volgende uit:

- maak een dictionary met 3 sleutel-waarde paren die iets over jou zeggen
- print de dictionary (1)
- voeg een extra sleutel toe
- print de dictionary (2)
- verwijder een sleutel
- print de dictionary (3)

Opdracht 3: CSV lezen

Maak twee bestanden aan in dezelfde map:

- `opdracht3.py`
- `data.csv` (zet hierin 3 regels met tekst)

In je Python bestand:

- importeer `csv` bovenaan
- maak een pad naar de map waar het CSV-bestand staat
tip: `file_path = path + "/" + file`
- open het CSV-bestand en lees alle regels
tip: gebruik `readlines()`
- print alle regels

Zorg er nu voor dat er komma's staan in je CSV-bestand (als deze er nog niet in staan). Ga daarna terug naar de Python bestand en voeg het volgende toe:

- een lege list voor regels die je gaat splitsen
- loop door elke regel en split op komma
tip:
for row in text:
 onderdelen = row.strip().split(',') # split op komma en haal overige spaties weg door de strip
 regels.append(onderdelen) # voeg de losse onderdelen toe aan de regels list
- print het eerste deel van elke regel (dus alles vóór de komma)
- print de eerste regel, laatste regel en het totaal aantal regels
tip: `regels[0]`, `regels[-1]`, `len(regels)`

Let op: vergeet niet ook het CSV-bestand in te leveren.

Les 2

Dit zijn de opdrachten van les 2.

Opdracht 1: Functies

Maak een nieuw Python-bestand opdracht1.py en voer het volgende uit:

- schrijf een functie die een lijst getallen ontvangt en het totaal teruggeeft
- schrijf een functie die het gemiddelde teruggeeft
- schrijf een functie die de hoogste waarde teruggeeft

Maak buiten je functies het volgende aan:

- een lijst met getallen

Roep vervolgens alle functies aan en print de resultaten.

Opdracht 2: Bestandsnamen ontleden

Maak een nieuw Python-bestand opdracht2.py en voer het volgende uit:

- Maak zelf een bestandsnaam als string, bijvoorbeeld: "data_gebruikers_v1.txt"
- Je hoeft dit tekstbestand niet echt aan te maken; je werkt alleen met de string.
- Verwijder de extensie .txt uit de bestandsnaam
tip: gebruik hiervoor de functie replace()
- Split de bestandsnaam op _ zodat je losse onderdelen krijgt
tip: gebruik hiervoor split()
- Print alle onderdelen afzonderlijk

Opdracht 3: Debug-challenge

Soms werkt je code gewoon: Python geeft geen foutmelding, alles draait netjes... maar de output klopt niet. Dat heet een logische fout. De code doet iets anders dan jij bedoelt.

Bij een logische fout:

- de code draait normaal
- Python klaagt niet
- maar de output is inhoudelijk fout

In deze opdracht ga je precies dát doen: je gaat op zoek naar een logische fout door goed naar de output te kijken, te ontdekken waarom de code verkeerd werkt en dit vervolgens zelf oplossen.

Maak een nieuw Python-bestand opdracht3.py en voer het volgende uit:

- gebruik deze foutcode:

```
def lees_data(bestand):
    data = []
    with open(bestand) as f:
        for regel in f:
            data.append(regel.split())
    return data

print(lees_data("data.csv"))
```

Maak een nieuw CSV-bestand data.csv in dezelfde map aan en voer het volgende uit:

- gebruik deze code:

```
naam,leeftijd,woonplaats
Yoana,18,Amersfoort
Becci,20,Zwolle
Norah,22,Den Haag
```

Run het Python-bestand opdracht3.py en voer het volgende uit:

- noteer de output die je krijgt
- beantwoord de volgende vragen:
 - welk scheidingsteken hoort een CSV te gebruiken, en welk scheidingsteken gebruikt de code nu?
 - aan welke plaatsnaam kun je zien dat de code het CSV-bestand verkeerd splitst, en waarom valt juist die plaatsnaam uit elkaar?
- vind de plek van de logische fout
- los het op
- print de correcte output

Let op: vergeet niet ook het CSV-bestand in te leveren.

Uitdagende Opdrachten

Opdracht 1: Mini-Integratie

Maak een bestand opdracht1.py en voer de volgende stappen uit:

- Maak een lijst met 5 zelfgekozen getallen.
Bijvoorbeeld: [12, 5, 33, 7, 19]
- Bereken het totaal van alle getallen. Sla dit op in een variabele totaal.
- Bereken het gemiddelde. Sla dit op in een variabele gemiddelde.
- Stop deze gegevens in een dictionary. De dictionary moet er ongeveer zo uitzien:

```
{
    "getallen": [...],
    "totaal": ...,
    "gemiddelde": ...
}
```

- Schrijf een functie die deze dictionary ontvangt en de informatie netjes print. Bijvoorbeeld:


```
print("Aantal getallen:", ...)
print("Totaal:", ...)
print("Gemiddelde:", ...)
```
- Roep de functie aan en print de resultaten.

Opdracht 2: Koppeling Python-onderdeel aan proces (niet verplicht)

Maak een bestand skill_library.py waarin je een informatiebibliotheek bouwt over alle Python-skills die je hebt geoefend in deze les. Je gebruikt hiervoor de skill-lijst (de code hiervoor staat op de volgende pagina):

CSV lezen → bestanden openen, regels lezen, splitten

Lists & Loops → lijst maken, itereren, berekeningen uitvoeren

Dictionaries → key-value opslag, data structureren

Functies → herbruikbare code, structuur aanbrengen

Bestandsnamen ontleden → string-bewerking, split + replace

Debug-challenge → foutanalyse, logisch denken

Mini-Integratie → combineren van meerdere Python-onderdelen in één script

Voeg het volgende toe aan `skill_library.py`:

- een dictionary met alle skills
maak een dictionary genaamd `skills` met deze structuur:

```
skills = {
    "CSV lezen": "bestanden openen, regels lezen, splitten",
    "Lists & Loops": "lijst maken, itereren, berekeningen uitvoeren",
    "Dictionaries": "key-value opslag, data structureren",
    "Functies": "herbruikbare code, structuur aanbrengen",
    "Bestandsnamen ontleden": "string-bewerking, split + replace",
    "Debug-challenge": "foutanalyse, logisch denken",
    "Mini-Integratie": "combineren van meerdere Python-onderdelen in één script"
}
```

- een functie die er als volgt uit ziet:

```
def uitleg(skill):
    print(f"Skill: {skill}")
    print(f"Wat je ermee oefent: {skills[skill]}")
```

- een functie die uitlegt waarom de skill belangrijk is (vul zelf verder in)

```
waarom = {
    "CSV lezen": "...",
    "Lists & Loops": "...",
    "Dictionaries": "...",
    "Functies": "...",
    "Bestandsnamen ontleden": "...",
    "Debug-challenge": "...",
    "Mini-Integratie": "..."
}
```

- voeg de volgende code toe aan je bestand:

```
def waarom_belangrijk(skill):
    print(f"Waarom belangrijk: {waarom[skill]}")

def toon_skill(skill):
    uitleg(skill)
    waarom_belangrijk(skill)
    print("-" * 40)

for skill in skills:
    toon_skill(skill)
```

- run de code
- beantwoord wat bovenstaand stukje code doet